

Modelando el mundo real con Kotlin

- De entidades físicas a estructuras de código.
- Diseño orientado a objetos para principiantes.



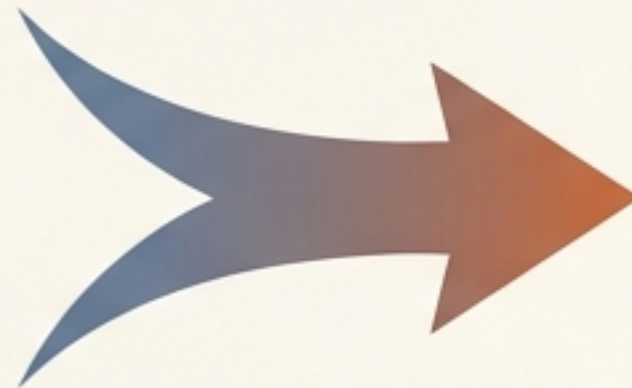
```
class SistemaBiblioteca {  
    val libros = mutableListOf<Libro>()  
    val usuarios = mutableListOf<Usuario>()  
  
    fun tomarLibroPrestado(usuario: Usuario,  
        libro: Libro) {  
    } // ...  
}  
  
data class Libro(val titulo: String,  
    val autor: String,  
    var estaDisponible: Boolean)
```


El caos de los datos dispersos

- Sin una estructura organizativa, los datos de un usuario son variables sueltas y difíciles de gestionar.
- Necesitamos una forma de empaquetar la identidad y la información
- Necesitamos una forma de empaquetar la identidad y la información en una sola unidad lógica.

```
val socio1Id =  
val socio1Id = "SOC-001"  
val socio1Nombre = "Juan"  
val socio1Nombre =
```

El Caos

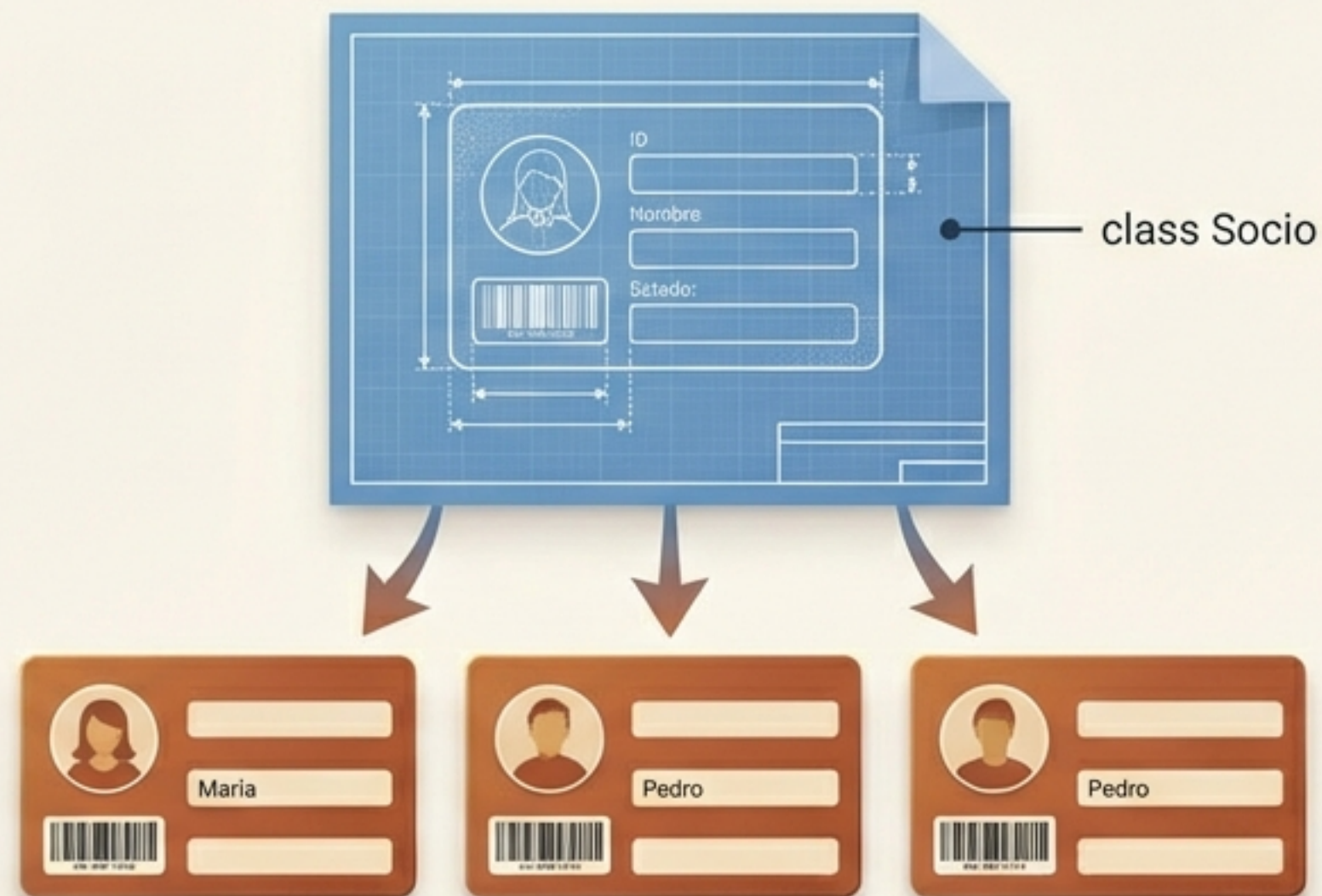
A user card representing a solution to the data chaos. It features a circular profile picture of a person with brown hair, a barcode at the bottom left with the text 'LEB-CARD-12345' below it, and three data fields on the right: 'ID del Socio:' with the value 'SOC-001', 'Nombre:' with the value 'Juan', and 'Estado:' with the value 'Activo'.

ID del Socio:	SOC-001
Nombre:	Juan
Estado:	Activo

La Solución

Agrupando información en un molde

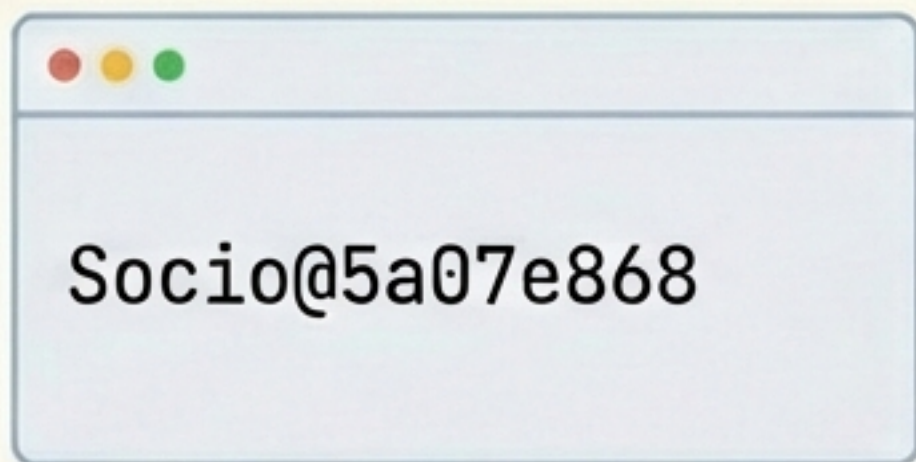
- Una class en Kotlin actúa como el plano de construcción.
- Define qué propiedades tendrá cada miembro que creemos.



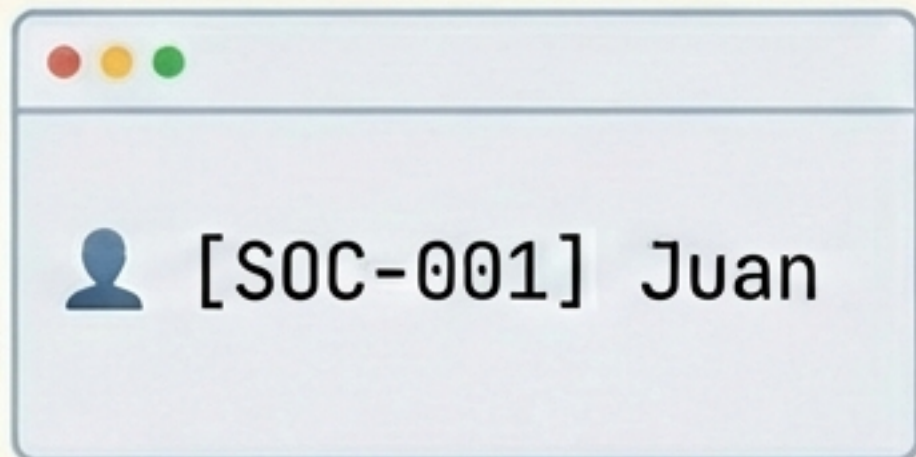
```
class Socio(val idSocio:String, val nombre:String)
```


Mostrando la información de nuestros objetos

- Por defecto, Kotlin imprime la dirección de memoria del objeto.
- Reescribimos la función `toString` para que el objeto se presente de forma humana.



Socio@5a07e868



 [SOC-001] Juan



```
override fun toString(): String {  
    return "👤 [$idSocio] $nombre"  
}
```


El problema de la repetición

- Libros, Revistas y DVDs comparten la mayoría de sus atributos básicos.
- ¿Escribimos el mismo código tres veces? Esto genera errores y dificulta el mantenimiento.



```
val id  
val titulo  
val autor  
val anio  
val copias  
val numeroPaginas
```



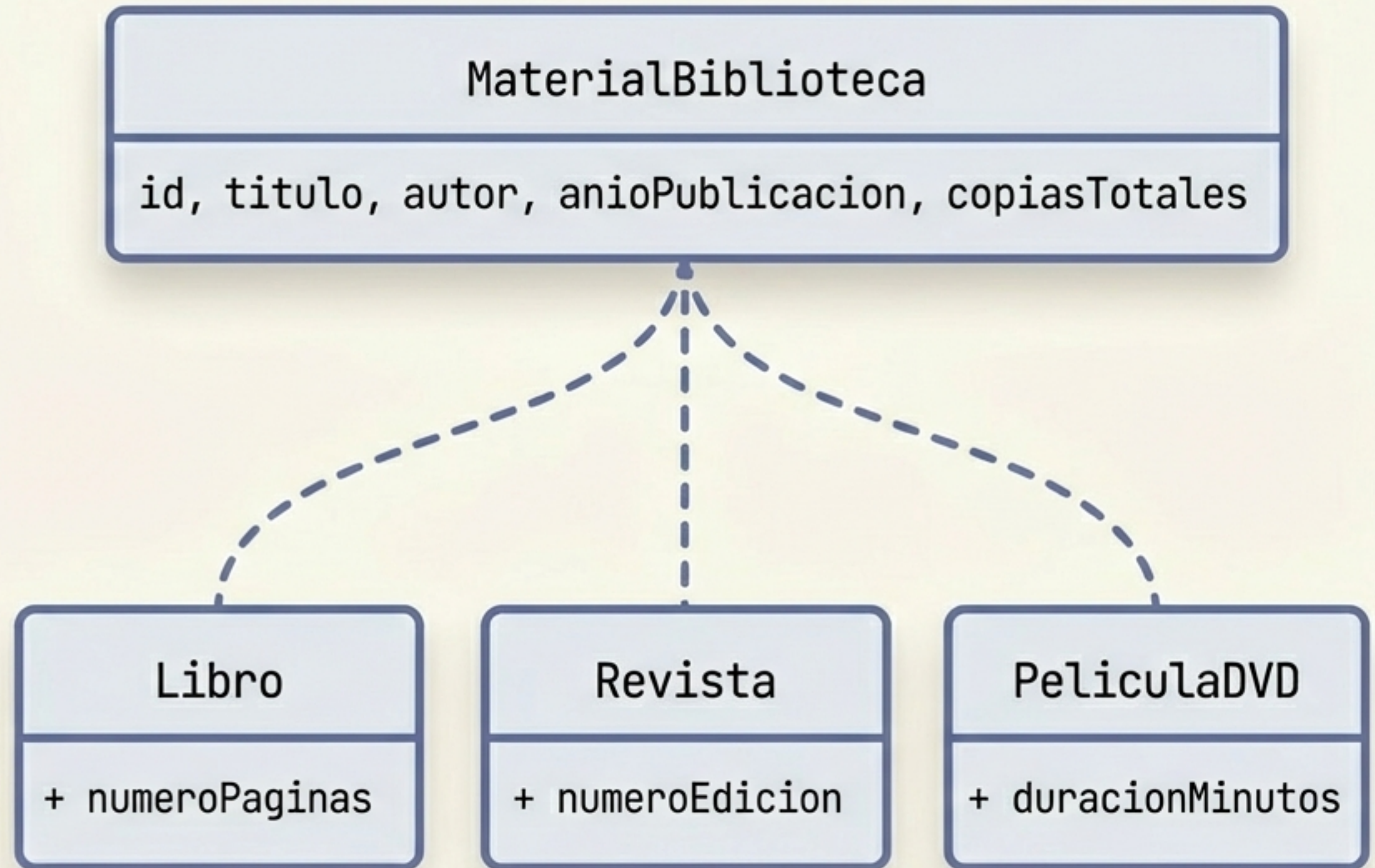
```
val id  
val titulo  
val autor  
val anio  
val copias  
val numeroEdicion
```



```
val id  
val titulo  
val autor  
val anio  
val copias  
val duracionMinutos
```


Extraer lo común para evitar la duplicación

- Principio DRY (Don't Repeat Yourself).
- Creamos una superclase que almacena los rasgos compartidos.
- Las clases hijas heredan automáticamente estas propiedades usando la sintaxis :



El concepto abstracto *versus* la realidad física

- Nadie puede pedir prestado un "Material Genérico". Solo puedes llevarte un Libro, un DVD o una Revista.
- La palabra clave `abstract` impide crear objetos directos de esta clase; solo sirve como plantilla base.



```
abstract class MaterialBiblioteca(\n    val id:String,\n    val titulo:String,\n    val autor:String,\n    val anioPublicacion:Int,\n    val copiasTotales:Int\n) {\n    var copiasDisponibles:Int = copiasTotales\n    // materialBiblioteca\n}
```



Dando vida al concepto con clases concretas

- La clase Libro hereda el molde abstracto y añade su rasgo único: las páginas.



```
class Libro(id: String, titulo:String, autor:String, anio:Int, copias:Int,  
    val numeroPaginas:Int) : MaterialBiblioteca(id, titulo, autor, anio, copias)
```



Diferentes formatos bajo una misma base

- Al delegar la información base a la superclase, nuestras clases concretas se mantienen limpias y enfocadas.



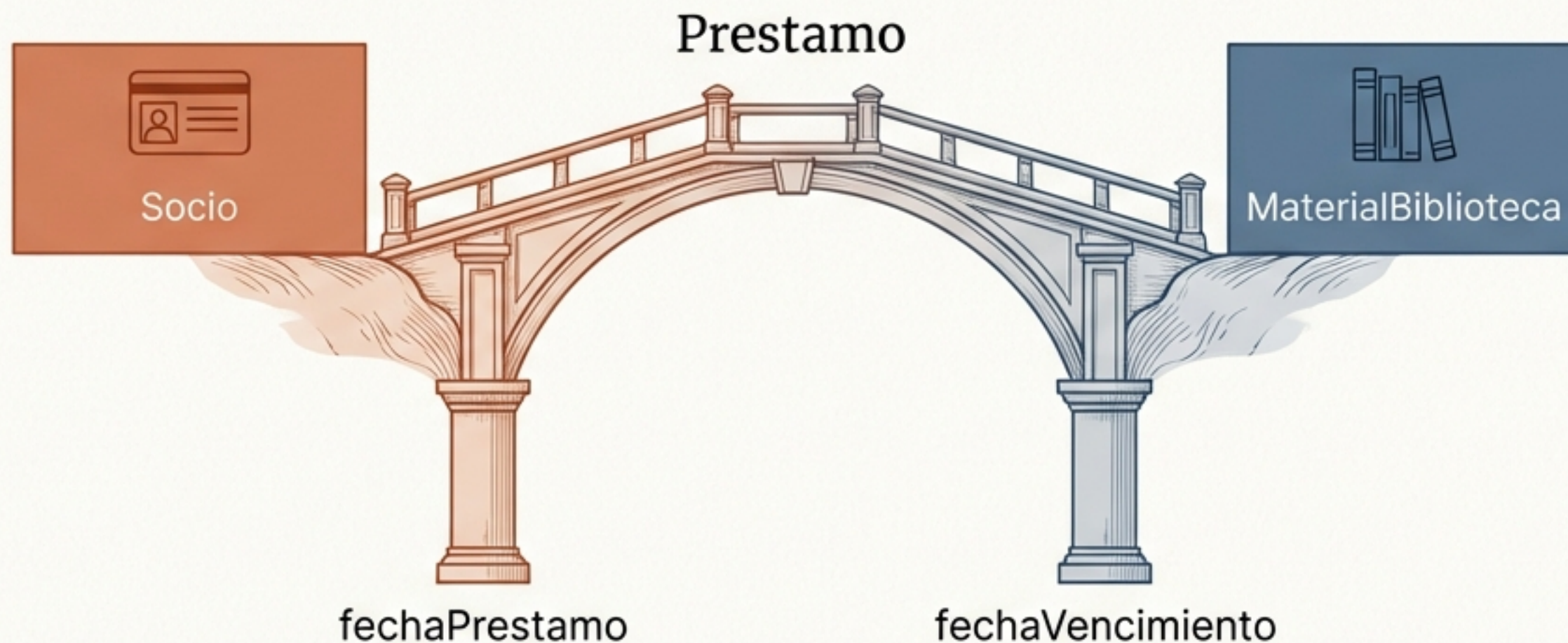
```
class Revista(  
    id:String,  
    titulo:String,  
    autor:String,  
    anio:Int,  
    copias:Int,  
    val numeroEdicion:Int  
) : MaterialBiblioteca(id, titulo,  
    autor, anio, copias)
```



```
class PeliculaDVD(  
    id:String,  
    titulo:String,  
    autor:String,  
    anio:Int,  
    copias:Int,  
    val duracionMinutos:Int  
) : MaterialBiblioteca(id, titulo,  
    autor, anio, copias)
```


Modelando acciones: El sistema de préstamos

- Las clases también pueden representar eventos que conectan otros objetos.



```
class Prestamo(val idSocio: String, val idMaterial:String,  
               val fechaPrestamo:LocalDate, val fechaVencimiento: LocalDate)
```


Los objetos conocen su propio estado

- Una clase no solo guarda datos, también contiene la lógica asociada a esos datos (métodos).
- El préstamo puede calcular por sí mismo si ha expirado.



```
fun estaVencido(): Boolean {  
    val hoy = LocalDate.now()  
    return hoy.isAfter(fechaVencimiento)  
}
```


El gran contenedor del ecosistema

- Necesitamos una clase central que actúe como gestor principal.
- Utiliza listas mutables (`mutableListOf`) para almacenar y administrar nuestros objetos.



```
class Biblioteca {  
    private val inventario = mutableListOf<MaterialBiblioteca>()  
    private val socios = mutableListOf<Socio>()  
    private val prestamos = mutableListOf<Prestamo>()  
}
```


Protegiendo las puertas de entrada

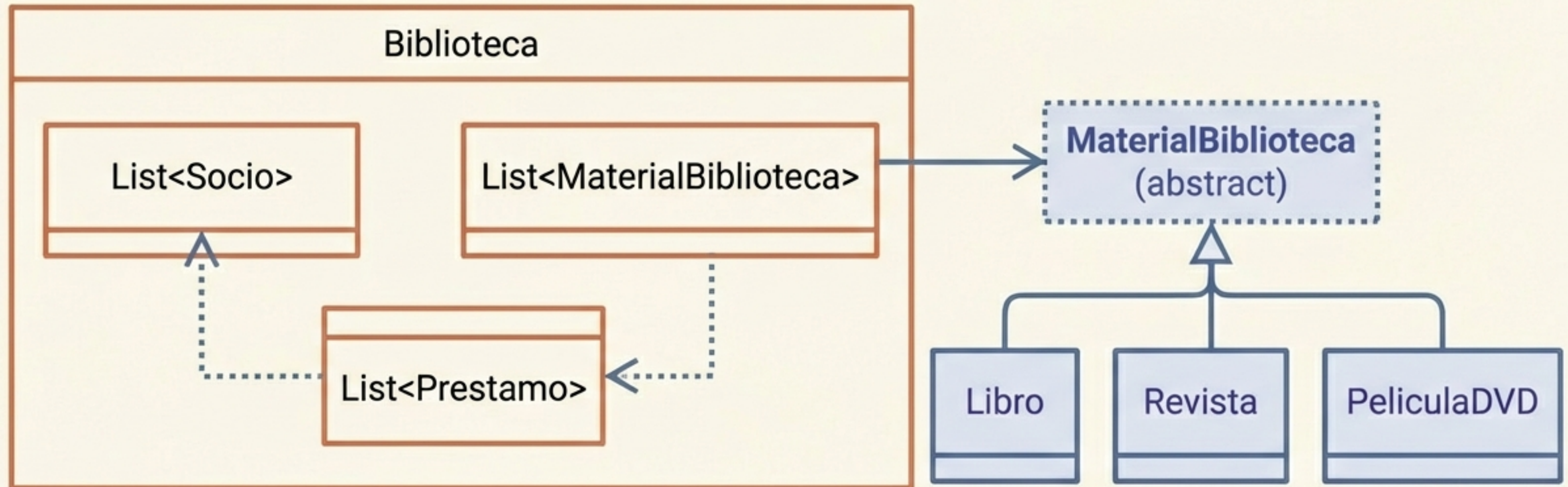
- Antes de instanciar nuestros objetos, debemos asegurar que los datos de entrada tengan el formato correcto.
- Usamos Expresiones Regulares (Regex) en funciones independientes para validar identificadores.



```
if (entrada.matches(Regex("SOC-[0-9]{3}")))  
    return entrada
```


El mapa arquitectónico del sistema

- Visión global de nuestro modelo de dominio.
- Las relaciones de herencia y composición trabajan juntas para simular la realidad operativa de la biblioteca.



Principios fundamentales para recordar



Agrupación lógica

Las clases empaquetan datos y comportamiento relacionados en una única entidad.



Clases abstractas

Modelan conceptos compartidos que no deben existir por sí solos en la realidad.



Herencia limpia

Reduce la duplicación de código compartiendo atributos desde una clase padre.



Depuración visual

Sobrescribir toString() transforma direcciones de memoria incomprensibles en información legible.